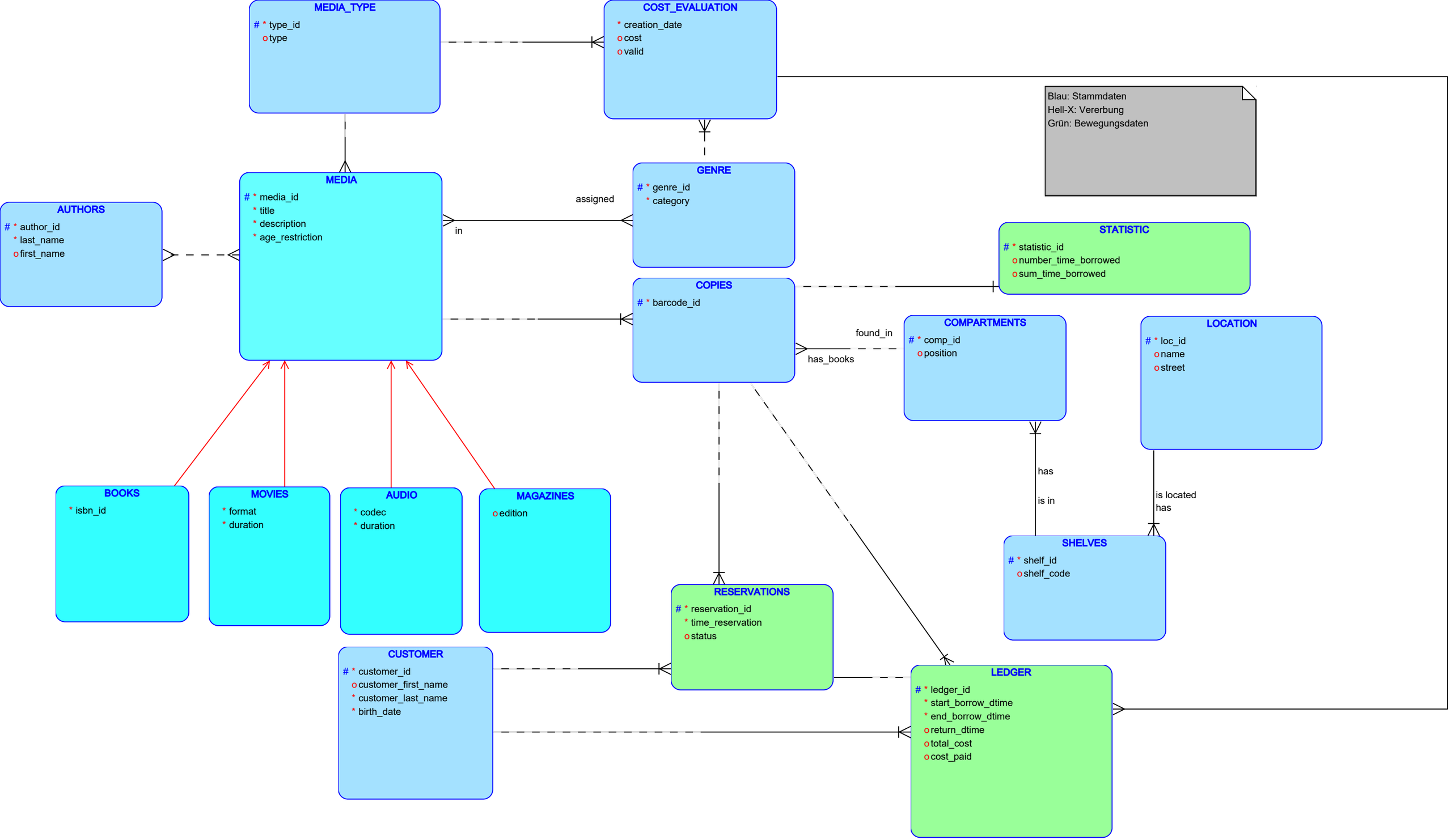
MEDIA_TYPE
* type_id
o type
COST_EVALUATION
* creation_date
o cost
o valid
Blau: Stammdaten
Hell-X: Vererbung
Grün: Bewegungsdaten
MEDIA
* media_id
* title
* description
* age_restriction
GENRE
* genre_id
* category
assigned
in
AUTHORS
* author_id
* last_name
o first_name
STATISTIC
* statistic_id
o number_time_borrowed
o sum_time_borrowed
COPIES
* barcode_id
found_in
has_books
COMPARTMENTS
* comp_id
o position
LOCATION
* loc_id
o name
o street
has
is in
is located
has
SHELVES
* shelf_id
o shelf_code
BOOKS
* isbn_id
MOVIES
* format
* duration
AUDIO
* codec
* duration
MAGAZINES
o edition
RESERVATIONS
* reservation_id
* time_reservation
o status
CUSTOMER
* customer_id
o customer_first_name
* customer_last_name
* birth_date
LEDGER
* ledger_id
* start_borrow_dtime
* end_borrow_dtime
o return_dtime
o total_cost
o cost_paid

INSERT statements

Customer:

```
INSERT INTO CUSTOMER (first_name, last_name, birth_date) VALUES
('Susanne','Lastowicka',TO_DATE('02.02.1902','DD.MM.YYYY'));
```

Media Type:

```
INSERT INTO FHS52423.MEDIA_TYPE

(MEDIA_TYPE_ID, NAME)

VALUES(0, 'BOOK');
```

MEDIA:

```
INSERT INTO FHS52423.MEDIA

(TITLE, DESCRIPTION, AGE_RESTRICTION, MEDIA_TYPE_ID)

VALUES('Star Wars Episode I', 'A long time ago in a galaxy far, far away', 6, 1);
```

MEDIA SUBType:

```
INSERT INTO FHS52423.BOOKS (MEDIA_ID, ISBN)

VALUES (

  (SELECT MEDIA_ID FROM FHS52423.MEDIA WHERE TITLE = '1984'),

  '9-9-9-9');
```

Author:

```
INSERT INTO FHS52423.AUTHORS

( FIRST_NAME, LAST_NAME)

VALUES('Sapphó', 'Ψάπφω');
```

WRITTEN_BY:

```
INSERT INTO FHS52423.WRITTEN_BY (AUTHOR_ID, MEDIA_ID)

VALUES (
```

(**SELECT** AUTHOR_ID **FROM** FHS52423.AUTHORS **WHERE** LAST_NAME = 'Orwell' **AND** FIRST_NAME = 'George'),

(**SELECT** MEDIA_ID **FROM** FHS52423.MEDIA **WHERE** TITLE = '1984'));

GENRE:

INSERT INTO FHS52423.GENRE

(NAME)

VALUES('SCI-FI');

Media-Genre:

INSERT INTO FHS52423.MEDIA_GENRE (MEDIA_ID, GENRE_ID)

VALUES (

(**SELECT** MEDIA_ID **FROM** FHS52423.MEDIA **WHERE** TITLE = 'Star Wars Episode I'),

(**SELECT** GENRE_ID **FROM** FHS52423.GENRE **WHERE** NAME = 'SCI-FI'));

Location:

INSERT INTO FHS52423.LOCATION

(NAME, STREET)

VALUES('Stadtbibliothek Bischofshofen', 'Kinostraße 4');

Shelf:

INSERT INTO FHS52423.SHELVES

(LOCATION_ID, SHELF_CODE)

VALUES(1, 'REGAL A1');

Compartment:

INSERT INTO FHS52423.COMPARTMENT

(SHELF_ID, "POSITION")

VALUES(1, 'Oben');

Copie:

```
INSERT INTO FHS52423.COPY (barcode, media_id, compartment_id)
VALUES (
  'BC001',
  (SELECT media_id FROM MEDIA WHERE title = 'Hiraeth'), 2);
```

COST_EVALUATION:

```
INSERT INTO FHS52423.COST_EVALUATION
(MEDIA_TYPE_ID, COST, VALID, CREATION_DATE, GENRE_ID)
VALUES(0, 1.5, 'Y', SYSDATE, 1);
```

LEDGER:

```
INSERT INTO FHS52423.LEDGER
(CUSTOMER_ID, COPY_ID, START_DATE, END_DATE, TOTAL_COST,
MEDIA_TYPE_ID, GENRE_ID)
VALUES (
  (SELECT customer_id FROM CUSTOMER WHERE last_name = 'Lux'),
  (SELECT copy_id FROM COPY WHERE barcode = 'BC004'),
  SYSDATE - 10,
  SYSDATE - 5,
  (SELECT MAX(COST) FROM COST_EVALUATION WHERE media_type_id = 0),
  0,
  (SELECT genre_id FROM COST_EVALUATION
   WHERE media_type_id = 0 AND cost = (SELECT MAX(cost) FROM
COST_EVALUATION WHERE media_type_id = 0)
   AND ROWNUM = 1));
```

Reservierung:

```
INSERT INTO FHS52423.RESERVATION (
  customer_id,
  copy_id,
  reservation_date,
```

```
    status
  )
  VALUES (
    (SELECT customer_id FROM CUSTOMER WHERE last_name = 'Lastowicka'),
    (SELECT copy_id FROM COPY WHERE barcode = 'BC004'),
    SYSDATE,
    'AKTIV');
```

Statistic:

```
INSERT INTO FHS52423.STATISTIC
(COPY_ID, NUMBER_TIME_BORROWED, SUM_TIME_BORROWED)
VALUES((SELECT copy_id FROM COPY WHERE barcode = 'BC004'), 1, 0);
```

Screenshot Beispiel Daten:

ID	KUNDE	MEDIUM_TITEL	BARCODE	TYP	GENRE	AUSGELIEHEN	RUECKGABE_SOLL	PREIS
2	Dominik Zach	Hiraeth	BC001	AUDIO	Dystopie	14.03.2026	18.03.2026	10 €
1	Alexander Lux	Star Wars Episode I	BC004	MOVIE	SCI-FI	13.03.2026	18.03.2026	1,5 €

MEDIA_ID	TITLE	DESCRIPTION	AGE_RESTRICTION	MEDIA_TYPE_ID
1	Star Wars Episode I	A long time ago in a galaxy far, far away	6	1
2	1984	1984, geschrieben von 1946 bis 1948	16	0
3	Hiraeth	homesickness	0	2
4	Vogue	Druckerzeugnis	0	3
7	TEST	test	18	0

CUSTOMER_ID	FIRST_NAME	LAST_NAME	BIRTH_DATE
1	Alexander	Lux	1900-01-01 00:00:00.000
2	Susanne	Lastowicka	1902-02-02 00:00:00.000
3	Dominik	Zach	1903-03-03 00:00:00.000
4	Max	Mustermann	2002-02-02 00:00:00.000

COPY_ID	BARCODE	MEDIA_ID	COMPARTMENT_ID	CREATED_AT
1	BC001	3	2	2026-03-23 17:59:53.000
3	BC002	1	2	2026-03-23 18:00:52.000
4	BC003	2	1	2026-03-23 18:00:56.000
5	BC004	1	4	2026-03-23 18:00:58.000

Für alle Insert Statements, DDL-Statements, Diagramme siehe:
https://github.com/LuxAlexander/Laboruebungen/tree/main/Datenbanksysteme

LIBRARY SCHEMA DDL

```
-- 1. SEQUENCES
CREATE SEQUENCE media_seq START WITH 1;
CREATE SEQUENCE author_seq START WITH 1;
CREATE SEQUENCE customer_seq START WITH 1;
CREATE SEQUENCE genre_seq START WITH 1;
CREATE SEQUENCE location_seq START WITH 1;
CREATE SEQUENCE shelf_seq START WITH 1;
CREATE SEQUENCE compartment_seq START WITH 1;
CREATE SEQUENCE copy_seq START WITH 1;
CREATE SEQUENCE ledger_seq START WITH 1;
CREATE SEQUENCE reservation_seq START WITH 1;
CREATE SEQUENCE statistic_seq START WITH 1;

-- 2. STAMMDATEN (BASE TABLES)
CREATE TABLE MEDIA_TYPE (
  media_type_id NUMBER PRIMARY KEY,
  name VARCHAR2(30) NOT NULL UNIQUE
);

CREATE TABLE GENRE (
  genre_id NUMBER PRIMARY KEY,
  name VARCHAR2(50) NOT NULL
);

CREATE TABLE AUTHORS (
  author_id NUMBER PRIMARY KEY,
```

```
  first_name VARCHAR2(50),
  last_name VARCHAR2(50) NOT NULL
);

CREATE TABLE CUSTOMER (
  customer_id NUMBER PRIMARY KEY,
  first_name VARCHAR2(50),
  last_name VARCHAR2(50) NOT NULL,
  birth_date DATE NOT NULL
);

-- 3. KOSTEN-MATRIX (COST_EVALUATION)
-- Hier ist der zusammengesetzte Primärschlüssel direkt integriert
CREATE TABLE COST_EVALUATION (
  genre_id NUMBER NOT NULL,
  media_type_id NUMBER NOT NULL,
  cost NUMBER(9,2),
  valid CHAR(1),
  creation_date DATE DEFAULT SYSDATE,
  CONSTRAINT pk_cost_matrix PRIMARY KEY (genre_id, media_type_id),
  CONSTRAINT ce_genre_fk FOREIGN KEY (genre_id) REFERENCES GENRE(genre_id),
  CONSTRAINT ce_media_type_fk FOREIGN KEY (media_type_id) REFERENCES
MEDIA_TYPE(media_type_id)
);

-- 4. MEDIA & SUBTYPES
CREATE TABLE MEDIA (
  media_id NUMBER PRIMARY KEY,
  title VARCHAR2(100) NOT NULL,
```

```
  description VARCHAR2(500),
  age_restriction NUMBER,
  media_type_id NUMBER NOT NULL,
  CONSTRAINT media_type_fk FOREIGN KEY (media_type_id) REFERENCES
MEDIA_TYPE(media_type_id)
);

CREATE TABLE BOOKS (
  media_id NUMBER PRIMARY KEY,
  isbn VARCHAR2(50) NOT NULL,
  CONSTRAINT books_media_fk FOREIGN KEY (media_id) REFERENCES
MEDIA(media_id)
);

CREATE TABLE AUDIO (
  media_id NUMBER PRIMARY KEY,
  codec VARCHAR2(20) NOT NULL,
  duration NUMBER(10) NOT NULL,
  CONSTRAINT audio_media_fk FOREIGN KEY (media_id) REFERENCES
MEDIA(media_id)
);

CREATE TABLE MOVIES (
  media_id NUMBER PRIMARY KEY,
  format VARCHAR2(20) NOT NULL,
  duration NUMBER(10) NOT NULL,
  CONSTRAINT movies_media_fk FOREIGN KEY (media_id) REFERENCES
MEDIA(media_id)
);
```

```
CREATE TABLE MAGAZINES (
   media_id NUMBER PRIMARY KEY,
   edition VARCHAR2(30),
   CONSTRAINT magazines_media_fk FOREIGN KEY (media_id) REFERENCES MEDIA(media_id)
);

-- 5. LAGERSTRUKTUR
CREATE TABLE LOCATION (
   location_id NUMBER PRIMARY KEY,
   name VARCHAR2(30),
   street VARCHAR2(50)
);

CREATE TABLE SHELF (
   shelf_id NUMBER PRIMARY KEY,
   location_id NUMBER NOT NULL,
   shelf_code VARCHAR2(30),
   CONSTRAINT shelf_location_fk FOREIGN KEY (location_id) REFERENCES LOCATION(location_id)
);

CREATE TABLE COMPARTMENT (
   compartment_id NUMBER PRIMARY KEY,
   shelf_id NUMBER NOT NULL,
   position VARCHAR2(30),
   CONSTRAINT compartment_shelf_fk FOREIGN KEY (shelf_id) REFERENCES SHELF(shelf_id)
);
```

```
CREATE TABLE COPY (
  copy_id NUMBER PRIMARY KEY,
  barcode VARCHAR2(50) UNIQUE NOT NULL,
  media_id NUMBER NOT NULL,
  compartment_id NUMBER NOT NULL,
  created_at DATE DEFAULT SYSDATE,
  CONSTRAINT copy_media_fk FOREIGN KEY (media_id) REFERENCES MEDIA(media_id),
  CONSTRAINT copy_compartment_fk FOREIGN KEY (compartment_id) REFERENCES COMPARTMENT(compartment_id)
);

-- 6. TRANSAKTIONEN (LEDGER & RESERVATION)
CREATE TABLE LEDGER (
  ledger_id NUMBER PRIMARY KEY,
  customer_id NUMBER NOT NULL,
  copy_id NUMBER NOT NULL,
  start_date DATE NOT NULL,
  end_date DATE,
  return_date DATE,
  total_cost NUMBER(9,2),
  cost_paid NUMBER(9,2),
  genre_id NUMBER,      -- Teil der Kostenverknüpfung
  media_type_id NUMBER,   -- Teil der Kostenverknüpfung
  CONSTRAINT ledger_customer_fk FOREIGN KEY (customer_id) REFERENCES CUSTOMER(customer_id),
  CONSTRAINT ledger_copy_fk FOREIGN KEY (copy_id) REFERENCES COPY(copy_id),
  CONSTRAINT fk_ledger_cost_matrix FOREIGN KEY (genre_id, media_type_id)
```

```
    REFERENCES COST_EVALUATION (genre_id, media_type_id)
);

CREATE TABLE RESERVATION (
  reservation_id NUMBER PRIMARY KEY,
  customer_id NUMBER NOT NULL,
  copy_id NUMBER NOT NULL,
  ledger_id NUMBER,
  reservation_date DATE NOT NULL,
  status VARCHAR2(20),
  CONSTRAINT reservation_customer_fk FOREIGN KEY (customer_id) REFERENCES CUSTOMER(customer_id),
  CONSTRAINT reservation_copy_fk FOREIGN KEY (copy_id) REFERENCES COPY(copy_id),
  CONSTRAINT reservation_ledger_fk FOREIGN KEY (ledger_id) REFERENCES LEDGER(ledger_id)
);

-- 7. STATISTIK
CREATE TABLE STATISTIC (
  statistic_id NUMBER PRIMARY KEY,
  copy_id NUMBER NOT NULL UNIQUE,
  number_time_borrowed NUMBER DEFAULT 0,
  sum_time_borrowed NUMBER DEFAULT 0,
  CONSTRAINT stat_copy_fk FOREIGN KEY (copy_id) REFERENCES COPY(copy_id)
);

-- 8. RELATIONEN (MANY-TO-MANY)
CREATE TABLE MEDIA_GENRE (
```

```
  media_id NUMBER,
  genre_id NUMBER,
  PRIMARY KEY (media_id, genre_id),
  CONSTRAINT mg_media_fk FOREIGN KEY (media_id) REFERENCES MEDIA(media_id),
  CONSTRAINT mg_genre_fk FOREIGN KEY (genre_id) REFERENCES GENRE(genre_id)
);

CREATE TABLE WRITTEN_BY (
  media_id NUMBER,
  author_id NUMBER,
  PRIMARY KEY (media_id, author_id),
  CONSTRAINT wb_media_fk FOREIGN KEY (media_id) REFERENCES MEDIA(media_id),
  CONSTRAINT wb_author_fk FOREIGN KEY (author_id) REFERENCES AUTHORS(author_id)
);

-- ======================
-- TRIGGERS (LOGIK)
-- ======================

-- Auto-Increment Triggers
CREATE OR REPLACE TRIGGER media_bi BEFORE INSERT ON MEDIA FOR EACH ROW BEGIN
  IF :NEW.media_id IS NULL THEN SELECT media_seq.NEXTVAL INTO :NEW.media_id FROM dual; END IF; END;
/
CREATE OR REPLACE TRIGGER author_bi BEFORE INSERT ON AUTHORS FOR EACH ROW BEGIN
```

```
  IF :NEW.author_id IS NULL THEN SELECT author_seq.NEXTVAL INTO :NEW.author_id FROM dual; END IF; END;

/

CREATE OR REPLACE TRIGGER customer_bi BEFORE INSERT ON CUSTOMER FOR EACH ROW BEGIN

  IF :NEW.customer_id IS NULL THEN SELECT customer_seq.NEXTVAL INTO :NEW.customer_id FROM dual; END IF; END;

/

CREATE OR REPLACE TRIGGER genre_bi BEFORE INSERT ON GENRE FOR EACH ROW BEGIN

  IF :NEW.genre_id IS NULL THEN SELECT genre_seq.NEXTVAL INTO :NEW.genre_id FROM dual; END IF; END;

/

CREATE OR REPLACE TRIGGER location_bi BEFORE INSERT ON LOCATION FOR EACH ROW BEGIN

  IF :NEW.location_id IS NULL THEN SELECT location_seq.NEXTVAL INTO :NEW.location_id FROM dual; END IF; END;

/

CREATE OR REPLACE TRIGGER shelf_bi BEFORE INSERT ON SHELF FOR EACH ROW BEGIN

  IF :NEW.shelf_id IS NULL THEN SELECT shelf_seq.NEXTVAL INTO :NEW.shelf_id FROM dual; END IF; END;

/

CREATE OR REPLACE TRIGGER compartment_bi BEFORE INSERT ON COMPARTMENT FOR EACH ROW BEGIN

  IF :NEW.compartment_id IS NULL THEN SELECT compartment_seq.NEXTVAL INTO :NEW.compartment_id FROM dual; END IF; END;

/

CREATE OR REPLACE TRIGGER copy_bi BEFORE INSERT ON COPY FOR EACH ROW BEGIN

  IF :NEW.copy_id IS NULL THEN SELECT copy_seq.NEXTVAL INTO :NEW.copy_id FROM dual; END IF; END;

/
```

```
CREATE OR REPLACE TRIGGER ledger_bi BEFORE INSERT ON LEDGER FOR EACH ROW BEGIN
  IF :NEW.ledger_id IS NULL THEN SELECT ledger_seq.NEXTVAL INTO :NEW.ledger_id FROM dual; END IF; END;
/
CREATE OR REPLACE TRIGGER reservation_bi BEFORE INSERT ON RESERVATION FOR EACH ROW BEGIN
  IF :NEW.reservation_id IS NULL THEN SELECT reservation_seq.NEXTVAL INTO :NEW.reservation_id FROM dual; END IF; END;
/
CREATE OR REPLACE TRIGGER statistic_bi BEFORE INSERT ON STATISTIC FOR EACH ROW BEGIN
  IF :NEW.statistic_id IS NULL THEN SELECT statistic_seq.NEXTVAL INTO :NEW.statistic_id FROM dual; END IF; END;
/

-- Business Logic Triggers
CREATE OR REPLACE TRIGGER customer_date_trunc BEFORE INSERT OR UPDATE ON CUSTOMER FOR EACH ROW BEGIN :NEW.birth_date := TRUNC(:NEW.birth_date); END;
/

CREATE OR REPLACE TRIGGER trg_update_statistic AFTER INSERT ON LEDGER FOR EACH ROW BEGIN
  UPDATE STATISTIC SET number_time_borrowed = number_time_borrowed + 1,
    sum_time_borrowed = sum_time_borrowed + (NVL(:NEW.return_date, SYSDATE) - :NEW.start_date)
  WHERE copy_id = :NEW.copy_id;
END;
/

-- ARC-CONSTRAINTS (Validierung der Medientypen)
```

```
CREATE OR REPLACE TRIGGER trg_books_check BEFORE INSERT OR UPDATE ON BOOKS FOR EACH ROW DECLARE v_type NUMBER; BEGIN

  SELECT media_type_id INTO v_type FROM MEDIA WHERE media_id = :NEW.media_id;

  IF v_type != 1 THEN raise_application_error(-20001, 'MEDIA is not type BOOK'); END IF; END;

/

CREATE OR REPLACE TRIGGER trg_audio_check BEFORE INSERT OR UPDATE ON AUDIO FOR EACH ROW DECLARE v_type NUMBER; BEGIN

  SELECT media_type_id INTO v_type FROM MEDIA WHERE media_id = :NEW.media_id;

  IF v_type != 2 THEN raise_application_error(-20002, 'MEDIA is not type AUDIO'); END IF; END;

/

CREATE OR REPLACE TRIGGER trg_movies_check BEFORE INSERT OR UPDATE ON MOVIES FOR EACH ROW DECLARE v_type NUMBER; BEGIN

  SELECT media_type_id INTO v_type FROM MEDIA WHERE media_id = :NEW.media_id;

  IF v_type != 3 THEN raise_application_error(-20003, 'MEDIA is not type MOVIE'); END IF; END;

/

CREATE OR REPLACE TRIGGER trg_magazines_check BEFORE INSERT OR UPDATE ON MAGAZINES FOR EACH ROW DECLARE v_type NUMBER; BEGIN

  SELECT media_type_id INTO v_type FROM MEDIA WHERE media_id = :NEW.media_id;

  IF v_type != 4 THEN raise_application_error(-20004, 'MEDIA is not type MAGAZINE'); END IF; END;

/
```